

SEGUIMIENTO DECISIONES DE ARQUITECTURA/DISEÑO DE SOFTWARE

Seguridad y Aislamiento de Roles (Acceso a Funcionalidades)

Problema/ Preocupación de diseño

El sistema DashKPI interactúa con múltiples actores (Administrador, Líder de proyecto, Miembro de equipo, Cliente/Usuario). Cada actor tiene acceso a un conjunto distinto de funcionalidades (e.g., el Administrador gestiona roles, el Líder planifica el sprint, el Miembro registra tiempo). La preocupación es el diseño de un mecanismo de **Autorización y Control de Acceso basado en Roles (RBAC)** que prevenga que usuarios con roles menores (como "Miembro de equipo") puedan acceder a funcionalidades administrativas o de liderazgo (como "Gestionar roles" o "Planificar sprint").

Orientación de la decisión

Implementación de un Modelo de Control de Acceso basado en Roles (RBAC). Hay que asegurar que las peticiones del usuario sean validadas contra su rol antes de permitir la ejecución de la funcionalidad. Los elementos que serán afectados son la **Capa de Seguridad/Autenticación** y la **Lógica de Negocio** dentro del **Sistema DashKPI**.

Supuestos

Se asume que el sistema ya cuenta con un mecanismo de **Autenticación** que verifica la identidad del usuario y asigna un rol único al inicio de la sesión.

Restricciones

* La validación de permisos debe ser rápida para no impactar la experiencia de usuario (latencia). * El modelo debe ser lo suficientemente flexible para soportar futuros roles o la combinación de permisos.

Requerimientos relacionados

Seguridad Funcional: Solo usuarios autorizados según su rol deben poder ejecutar operaciones críticas. **Requerimiento de Auditoría:** El sistema debe poder registrar los intentos de acceso no autorizados.

Alternativas consideradas

Alternativa 1 RBAC Basado en Anotaciones/Decoradores (A nivel de Código): Usar decoradores o anotaciones en el código de la API/controlador que verifiquen el rol requerido antes de ejecutar el método

Alternativa 2 RBAC Basado en Interceptores/Middleware (A nivel de Framework): Implementar un módulo de *middleware* o un interceptor que verifique la ruta de acceso contra el rol del usuario antes de que la petición llegue a la lógica de negocio.

Alternativa n ABAC (Attribute-Based Access Control): Usar un sistema más dinámico que evalúe múltiples atributos (rol, tiempo, proyecto) para determinar el acceso.

Evaluación de alternativas	Alternativa 1 Ventajas: Es simple y granular; el permiso está claramente asociado a la función. Desventajas: Puede llevar a código repetitivo (<i>boilerplate</i>) en muchas funciones y es fácil de olvidar en nuevas implementaciones. Alternativa 2 Ventajas: Centraliza la lógica de seguridad, evitando código repetitivo. Es más fácil de mantener y auditar. Desventajas: Puede ser más complejo de configurar inicialmente en el <i>framework</i>. Alternativa n Ventajas: Máxima flexibilidad para escenarios complejos. Desventajas: Sobrecarga de diseño y complejidad innecesaria para los roles bien definidos del diagrama (Administrador, Líder, Miembro).
Alternativa seleccionada y Justificación	Alternativa 2: RBAC Basado en Interceptores/Middleware. Centraliza la validación de seguridad fuera de la lógica de negocio, lo que mejora la mantenibilidad, reduce el riesgo de errores por omisión, y permite una gestión de permisos más clara y escalable.
Decisiones relacionadas	Se decide mantener los roles estáticos según el diagrama de contexto (Administrador, Líder de proyecto, Miembro de equipo, Cliente/Usuario) a menos que se demuestre una necesidad de mayor granularidad.
Comentarios	El rol de Cliente/Usuario debe ser cuidadosamente diseñado ya que tiene la función de "Aplicar filtros en reportes" y "Visualizar KPIs", lo que implica acceso a información sensible.